

Using Transformers Cheat Sheet

1. Transformers nedir?

Transformers, metin gibi veriler üzerinde önceden eğitilmiş modelleri (pretrained models) kullanmamıza sağlayan bir kütüphanedir. Hugging Face, bu modelleri yüklemeyi ve farklı doğal dil işleme (Natural Language Processing - NLP) görevlerinde kullanmayı kolaylaştırır.

Bir modeli kullanırken genellikle adımlar vardır:

1. Preprocessing (eğilim): Girdi metni modelin anlayacağı formata dönüştürme.
2. Inference (çıkarma): Modelin girdiyi işleyip sonucu üretmesi.
3. Postprocessing (son işleme): Model çıktısını okunabilir bir sonuca dönüştürme.

Pipeline nasıl çalışır?

pipeline, bu adımları tek bir array üzerinden çalıştırmamıza sağlar. Başlangıç seviyesinde modelin ve tokenizer'ın ayrıntılarıyla tek tek uğraşmadan sonuç alabiliriz.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
result = classifier("I love learning about language models!")
print(result)
```

Kod ne yapıyor?

`pipeline`: Transformers kütüphanesinden pipeline aracını e aktarmak.
`"sentiment-analysis"`: Yapılacak görevin duygu analizi olduğunu belirtir.
`classifier(...)`: Verilen cümleyi analiz eder.
`print(result)`: Sonucu ekrana yazdırır. Çıktıda genellikle bir etiket (label) ve görevin skoru (score) bulunur.

Kısaca: Pipeline, metni hazırlar, modele gönderir ve sonucu düzenleyerek verir.

2. Model ve Tokenizer

Modeli doğrudan metinle çalıştırmak o zaman mümkün değildir. Metnin önce sayısal temsile dönüştürülmesi gerekir. Bu noktada tokenizer ve model birlikte çalışır.

Tokenizer (belirteleyici): Metni token adı verilen parçalara ayırır ve bunları sayısal kimliklere (token IDs) dönüştürür.

Model: Bu sayısal girdileri işler ve görevine uygun çıktılar üretir.

Checkpoint (model kaydı): Modelin ayrıntılarına ve ilgili yapılandırma dosyalarını içeren kayıt.

Pretrained model (önceden eğitilmiş model): Daha önce büyük miktarda veriyle eğitilmiş ve yeniden kullanılabilen model.

Tokenizer'ı yüklemek

```
from transformers import AutoTokenizer

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

inputs = tokenizer("I love learning about NLP!")
print(inputs)
```

Kod ne yapıyor?

`AutoTokenizer`: Verilen checkpoint'e uygun tokenizer sınıfıdır.

`from_pretrained(checkpoint)`: İlgili tokenizer ayarlarını indirir veya kayıtlı serimlenmişler.

`tokenizer(...)`: Metni token ID'leri gibi modelin kullanacağı girdilere dönüştürür.

`inputs`: Genellikle `input_ids` ve modele göre `attention_mask` gibi alanları içeren bir dizidir.

Tokenizer ile ilgili temel terimler

Token: Metnin model tarafından işlenen parçası. Bir token kelimenin tamamı, kelimenin bir bölümü veya noktalama işareti olabilir.

Token ID: Token'ın sayısal karşılığı.

Special tokens (özel token'lar): Modelin girdiyi anlamlandırmasına yardımcı olan özel işaretler.

Vocabulary (kelime/token dizisi): Token'larla bunların ID'leri arasındaki eşleştirme.

Tokenizer'ın kullandığı parçalama yöntemi modele göre değişebilir. Yaygın yaklaşımlar arasında WordPiece, BPE (Byte-Pair Encoding) ve Unigram bulunur.

Faydalı tokenizer yöntemleri

```
tokens = tokenizer.tokenize("I love learning about NLP!")
ids = tokenizer.convert_tokens_to_ids(tokens)
text = tokenizer.decode(ids)
```

```
print(tokens)
print(ids)
print(text)
```

`tokenize()`: Metni token parçalarına ayırır.

`convert_tokens_to_ids()`: Token parçalarını sayısal ID'lere çevirir.

`decode()`: ID'leri yeniden okunabilir metne dönüştürür.

Not: Decode edilen metin, özel token'lar ve tokenizer'ın kuralları nedeniyle ilk yazılan metinle her zaman karakter karakter aynı olmayabilir.

3. Model türleri: AutoModel ve görev başlıklar

Transformers'ta farklı ihtiyaçları için farklı modeller vardır.

AutoModel: Temel Transformer modelini yapar. Genellikle gizli durumlar (hidden states) gibi ara temsiller üretir.

AutoModelForSequenceClassification: Metin sınıflandırma görevleri için sınıflandırma başlığı (classification head) eklenmiş modeli yapar.

Classification head (sınıflandırma başlığı): Temel modelin temsilini sınıf tahmini için kullanılabilecek katkıda bulunan katman.

Duygu analizi gibi bir sınıflandırma görevi için görev odaklı sınıf kullanılır:

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

inputs = tokenizer("I love learning about NLP!", return_tensors="pt")
outputs = model(**inputs)

print(outputs.logits)
```

Kod ne yapıyor?

Tokenizer ve sınıflandırma modeli aynı checkpoint üzerinden yüklenir.

`return_tensors="pt"` girdileri PyTorch tensörleri biçiminde döndürür. (pt, PyTorch anlamına gelir.)

```
model(**inputs) s z l kteki girdileri modele ayr arg manlar olarak iletir.  
outputs.logits, s n fland rma s n flar i in modelin retti i ham skorlar i erir.
```

Logits ve olas l k

Logits, modelin do rudan retti i ham s n f skorlar d r; tek ba na olas l k olarak yorumlanmamal d r. Softmax, bu skorlar toplam 1 olan olas l k de erlerine d n t rme i in kullan labilir.

```
import torch  
  
probabilities = torch.softmax(outputs.logits, dim=-1)  
print(probabilities)
```

Burada `dim=-1`, i lemin son boyut zerinde yap laca n belirtir. S n fland rmada bu boyut genellikle s n f skorlar n i erir.

nemli ayr m: `AutoModel` ile `AutoModelForSequenceClassification` ayn ey de ildir. Birincisi temel temsilleri, ikincisi ise s n fland rma g revine y nelik kt lar verir.

4. Hidden states (gizli durumlar) ve kt boyutlar

Temel Transformer modeli, girdideki token'lar i in say sal temsiller retir. Bunlara hidden states (gizli durumlar) denir.

Bir modelin `last_hidden_state` kt s n n boyutlar genellikle u s radad r:

```
[batch_size, sequence_length, hidden_size]
```

`batch_size` (toplu i boyutu): Ayn anda i lenen rnek say s .

`sequence_length` (dizi uzunlu u): Girdideki token say s .

`hidden_size` (gizli katman boyutu): Her token i in retilen temsilin say sal uzunlu u.

rne in ekil `[2, 10, 768]` olsayd , bu iki rnek, her rnekte 10 token ve her token i in 768 boyutlu temsil oldu u anlam na gelirdi. Bu yaln zca boyutlar a klayan bir rnektir; her modelde bu say lar ayn de ildir.

5. Birden fazla metin: batching, padding ve truncation

Batching (toplu i leme), birden fazla girdiyi tek seferde modele vermektir. Bu i lem verimlili i art rabilir. Ancak metinlerin uzunluklar farkl olabilece i i in girdilerin uygun bi imde d zenlenmesi gerekir.

Padding (dolgu)

Bir batch i indeki dizileri ayn uzunlu a getirmek i in k sa dizilerin sonuna zel dolgu token'lar eklenebilir. B ylece girdiler ayn boyutta bir tens r h line getirilebilir.

Truncation (k rpma)

Modelin kabul edebilece i uzunluk s n r n a an girdilerin belirli bir uzunlukta kesilmesidir. ok uzun girdilerde kullan labilir; ancak kesilen b l mdeki bilgi kaybolabilir.

```
batch = [  
    "I love NLP.",  
    "Transformers help me work with language."  
]  
  
inputs = tokenizer(  
    batch,  
    padding=True,  
    truncation=True,  
    return_tensors="pt"
```

)

Kod ne yapıyor?

`batch`: ki metinden oluşan bir liste.
`padding=True`: Batch içindeki dizileri aynı uzunluğa getirir.
`truncation=True`: Gerekirse uzun girdileri modelin sınırlarına göre kısaltır.
`return_tensors="pt"`: Sonuçları PyTorch tensörleri biçiminde döndürür.

Attention mask (dikkat maskesi)

`attention_mask`, modelin hangi token'lar dikkate alınacağını belirtmeye yardımcı olur. Yaygın kullanımda:

- 1: Gerçek girdi token'ı.
- 0: Padding ile eklenen token.

Modelin ve tokenizer'ın uygulamasına göre ayrıntılar değişebilir; bu yüzden maskeyi girdinin bir parçası olarak düşünmek en güvenli yaklaşımdır.

6. Bütün adımları birleştirmek

Aşağıdaki örnekte metin önce tokenizer ile hazırlanır, ardından sınıflandırma modeline verilir ve modelin ham skorları alınır.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

text = "This course is useful."
inputs = tokenizer(text, return_tensors="pt")
outputs = model(**inputs)

print(outputs.logits)
```

Aşağıdaki adımlar:

1. Checkpoint belirlenir.
2. Checkpoint'e uygun tokenizer ve görev modeli yüklenir.
3. Metin token ID'lerine dönüştürülür.
4. Girdiler modele gönderilir.
5. Model sınıflandırma için ham skorları verir.

Bu örnekte `logits` yazdırılır; doğrudan pozitif veya negatif etiketi yazdırmaz. Etiketleri görmek için çıktıların ayrıca yorumlanması ya da daha üst seviyeli `pipeline` kullanılması gerekir.

7. Modeli kaydetme ve yeniden yükleme

`save_pretrained()`, modelin veya tokenizer'ın gerekli dosyalarını bir klasöre kaydetmek için kullanılır. Böylece daha sonra aynı kayıt tekrar yüklenebilir.

```
model.save_pretrained("./my_model")
tokenizer.save_pretrained("./my_model")
```

Bu kod modelin ve tokenizer'ın dosyalarını `my_model` klasörüne kaydeder. Daha sonra aynı klasör yolu `from_pretrained()` içinde kullanılarak kayıtlı model yeniden yüklenebilir.

8. Inference (Kar m) ve da t m

Modelin e itilmi h lini tahmin retmek i in kullanmaya inference (kar m) denir. Modeli bir uygulama veya hizmet zerinden kullan ma a mak ise deployment (da t m) olarak adland r l r.

Hugging Face b l m nde ge en baz ara lar:

TGI (Text Generation Inference): Metin retimi modellerini servis etmek i in kullan lan ara .

vLLM: B y k dil modellerinde verimli kar m ve servis sunmaya odaklanan ara .

llama.cpp: zellikle yerel ortamda al t rma ve farkl donan mlarda kar m yapma senaryolar nda kullan lan proje.

Ara se imi; kullan lacak model, donan m, performans ihtiyac ve da t m ortam na ba l d r. Bu ara lar ayn ey de ildir; farkl kullan m ihtiya lar na hitap ederler.

Metin retiminde kar la labilecek parametreler

`max_new_tokens`: retilecek yeni token say s na st s n r koyar.

`min_new_tokens`: retilecek yeni token say s i in alt s n r belirler.

`temperature`: retimin ne kadar rastlant sal olaca n etkiler. D k de erler genellikle daha az e itlilik getirir.

`top_k`: Her ad mda de erlendirmeye al nan en y ksek skorlu token se eneklerini s n rlar.

`top_p`: Olas l k toplam na g re aday token k mesini s n rlar.

Stop sequence (durdurma dizisi): retimin belirli bir metin dizisine ula t nda durmas n sa layabilir.

Repetition/frequency/presence penalty (tekrar s kl /varl cezas): Tekrarlar azaltmak veya retim davran n y nlendirmek i in kullan lan ayarlar.

Bu parametrelerin t m her ara ta ayn adla veya ayn bi imde bulunmayabilir. Kullan lan k t phanenin ayarlar kontrol edilmelidir.